Kerberos at the Library/Implementation Level: A Production Engineer’s Guide
Section 1: The 80/20 Core (Hiring-Critical)
1. Credential Cache Lifecycle & Storage Mechanisms
Why hiring managers care: 90% of “auth randomly stopped working” tickets trace to credential cache misunderstandings. Engineers who can’t debug cache issues waste days on problems that take minutes to fix.
The reality:
· Credential caches (ccache) are not just a ticket store—they’re a state machine with acquisition, renewal, and expiration phases
· Default location: FILE:/tmp/krb5cc_${UID} (ancient, dangerous, still ubiquitous)
· Modern alternatives: KEYRING:persistent:${UID}, KCM:, DIR:/run/user/${UID}/krb5cc
Critical library behaviors:

What actually happens when you kinit

kinit user@REALM
→ libkrb5 contacts KDC (via UDP/TCP 88, fallback logic matters)
→ Gets TGT encrypted with user's key (derived from password)
→ Stores in ccache with:
- Start time (authtime)
- End time (endtime, typically 10hrs)
- Renewable-until time (typically 7 days)
→ Sets KRB5CCNAME env var (or relies on default)

What breaks in production:
· Process inheritance: Child processes inherit KRB5CCNAME, but the file might get deleted/rotated
· Renewal vs re-auth: kinit -R renews if you’re within renewable lifetime—but fails silently after that window
· Parallel cache corruption: Two kinit processes writing FILE: cache simultaneously = garbage bytes
· Insufficient permissions: Cache in /tmp with wrong perms = GSSAPI failures that say “No credentials” instead of “Permission denied”
Hiring signal:
· Junior: “I run kinit when auth breaks”
· Senior: “I check if the process has the right KRB5CCNAME, whether the cache is expired vs renewable, and if the KDC is even reachable before assuming credential issues”
Documentation:

· https://web.mit.edu/kerberos/krb5-latest/doc/basic/ccache_def.html
· https://web.mit.edu/kerberos/krb5-latest/doc/admin/conf_files/krb5_conf.html#libdefaults

1. Keytabs: The Production Auth Primitive
   Why hiring managers care: Services can't enter passwords. Engineers who don't understand keytabs can't deploy production services that need Kerberos.
   The mental model shift:
   · Password auth = user proves they know a secret
   · Keytab auth = service proves possession of a key (no password involved)
   Critical details:
   · Keytabs are binary files containing (principal, kvno, enctype, key) tuples
   · Created with: kadmin: ktadd -k /path/to/service.keytab service/hostname@REALM
   · kvno (key version number) is the IPC synchronization primitive between KDC and service
   · When you change a password/key, kvno increments—old tickets become invalid instantly
   What production engineers must know:

Viewing keytab contents

klist -ket /etc/krb5.keytab
Keytab name: FILE:/etc/krb5.keytab
KVNO Principal

```
3 host/server.example.com@EXAMPLE.COM (aes256-cts-hmac-sha1-96)
3 host/server.example.com@EXAMPLE.COM (aes128-cts-hmac-sha1-96)
```

Common production failures:
1. Keytab/KDC kvno mismatch: Service has kvno=3, KDC has kvno=4 → all auth fails with "Decrypt integrity check failed"
2. Wrong permissions: Keytab readable by non-service user = security hole, but unreadable by service = "Key table file not found"
3. Missing enctypes: Client negotiates aes256, keytab only has des3 → "KDC has no support for encryption type"
4. Hostname mismatches: Keytab says host/shortname@REALM, clients request host/fqdn.example.com@REALM → name canonicalization hell
The genius move:

Test keytab WITHOUT deploying service

kinit -kt /etc/krb5.keytab host/server.example.com@EXAMPLE.COM
klist # Should show valid TGT for service principal

Hiring signal:
· Junior: “Keytabs are like password files”
· Senior: “Keytabs are versioned symmetric keys that must stay synchronized with the KDC, and I always verify kvno and enctype support before deploying”
Documentation:
· https://web.mit.edu/kerberos/krb5-latest/doc/basic/keytab_def.html
· https://web.mit.edu/kerberos/krb5-latest/doc/admin/admin_commands/ktutil.html

1. Clock Skew: The Silent Killer
   Why hiring managers care: Kerberos will silently fail if clocks are wrong. This causes multi-hour outages where everything “looks fine” but nothing works.
   The hard truth:
   · Default tolerance: 5 minutes (300 seconds)
   · Enforced on: Ticket timestamps, authenticator timestamps, TGS requests
   · Not configurable in tickets—it’s a KDC policy
   What actually happens:

Client time: 14:05:00
KDC time: 14:00:00
Skew: 5 minutes (EXACTLY at threshold)

Result: 50/50 whether auth succeeds (depends on subsecond timing)

Real production scenarios:
1. VM snapshots: Restore snapshot from 2 days ago → all Kerberos auth fails until NTP syncs (can take minutes)
2. Container time drift: Container clock drifts +6 minutes → apps using Kerberos can’t auth, but HTTP/SSH work fine
3. Timezone vs UTC confusion: System shows correct local time, but UTC is wrong → Kerberos uses UTC internally
Library behavior you must know:

```
// From MIT Kerberos source (krb5_us_timeofday)
// Kerberos uses POSIX timestamp (seconds since epoch)
// Comparisons are ALWAYS in UTC
if (abs(client_time - server_time) > max_skew) {
return KRB5KRB_AP_ERR_SKEW; // "Clock skew too great"
}
```

The debug workflow:

Check local time in UTC

date -u

Check KDC time (requires access)

ssh kdc "date -u"

Check if clock skew is the issue

kinit user@REALM

If you see: "Clock skew too great while getting initial credentials"

→ Fix NTP, don't waste time debugging Kerberos

Counterintuitive case:
· Client and KDC clocks are both wrong by the same amount → auth works fine
· Client clock is correct, KDC clock is wrong by 2 minutes → auth fails
· Lesson: It’s relative skew that matters, not absolute accuracy
Hiring signal:
· Junior: “I’ll fix the clock”
· Senior: “I’ll check if this is actually clock skew (KRB5KRB_AP_ERR_SKEW) vs other issues, verify NTP is running, and confirm UTC time specifically since Kerberos doesn’t care about timezones”

Documentation:
· https://web.mit.edu/kerberos/krb5-latest/doc/admin/troubleshoot.html#clock-skew

1. KDC Discovery: How Libraries Actually Find the KDC
   Why hiring managers care: “Can’t contact KDC” is the #1 Kerberos error message.
   Engineers who don’t understand discovery mechanisms debug in the wrong direction.
   The precedence order (MIT Kerberos):

   1. /etc/krb5.conf [realms] section: Explicit kdc = kdc.example.com:88
   2. DNS SRV records: _kerberos._udp.REALM and _kerberos._tcp.REALM
   3. DNS TXT records: _kerberos.REALM (legacy, rarely used)
   4. Fallback logic: UDP port 88, then TCP port 88 if UDP times out
      Critical library behaviors:

Check what the library will actually do

KRB5_TRACE=/dev/stderr kinit user@REALM 2>&1 | grep -i kdc

You'll see:

[2847] 1706825901.471868: Getting initial credentials for user@REALM
[2847] 1706825901.471923: Sending request (169 bytes) to REALM
[2847] 1706825901.471935: Resolving hostname kdc1.example.com
[2847] 1706825901.472105: Sending initial UDP request to dgram 10.0.1.5:88
[2847] 1706825901.485234: Received answer (584 bytes) from dgram 10.0.1.5:88

Production failure modes:
1. DNS round-robin breaks stickiness: Multiple KDC IPs, client picks different KDC for TGT vs TGS → different kvno = decrypt failures
2. UDP fragmentation: Large tickets (many group memberships) exceed UDP MTU → silent packet drops, TCP fallback fails due to firewall
3. /etc/hosts interference: Library resolves “kdc.example.com” to wrong IP because /etc/hosts overrides DNS
4. Realm name case sensitivity: krb5.conf says EXAMPLE.COM, DNS has _kerberos._udp.example.com → lookup fails (realm names are case-sensitive in DNS queries)
The genius debug technique:

Bypass DNS and force specific KDC

```
cat > /tmp/krb5.conf <<EOF
[libdefaults]
default_realm = EXAMPLE.COM
[realms]
EXAMPLE.COM = {
kdc = 10.0.1.5:88
admin_server = 10.0.1.5:749
}
EOF

KRB5_CONFIG=/tmp/krb5.conf kinit user@EXAMPLE.COM
```

If this works but normal config doesn't → DNS/discovery issue

If this also fails → KDC or network issue

Hiring signal:
· Junior: “The KDC is down”
· Senior: “I’ll verify DNS SRV records, check if the library is resolving the right IP with KRB5_TRACE, test UDP vs TCP separately, and confirm the KDC is actually listening before assuming it’s down”
Documentation:
· https://web.mit.edu/kerberos/krb5-latest/doc/admin/realm_config.html#kdc-discovery
· https://web.mit.edu/kerberos/krb5-latest/doc/admin/conf_files/krb5_conf.html#realms

1. Service Principal Names (SPNs) and Name Canonicalization
   Why hiring managers care: Service auth fails more often due to SPN mismatches than any other reason. Engineers who don’t understand canonicalization waste days on “worked on my laptop” problems.
   The core concept:
   · Clients request tickets for service/hostname@REALM
   · hostname must match what the service expects (in its keytab)

· Libraries perform DNS canonicalization by default

Critical library behavior:

What happens when you do:

curl --negotiate -u : https://webapp.example.com/api

libcurl → GSSAPI → libkrb5:

1. Resolve "webapp.example.com" to canonical hostname (DNS A record → PTR lookup)
2. Canonical name might be "webapp-prod-1a.internal.example.com"
3. Request ticket for HTTP/webapp-prod-1a.internal.example.com@REALM
4. Service keytab has HTTP/webapp.example.com@REALM
5. Auth fails with "Server not found in Kerberos database"

krb5.conf controls that matter:

[libdefaults]
rdns = false # Don't do reverse DNS (PTR) lookup
dns_canonicalize_hostname = false # Don't canonicalize at all (use provided name)

Production anti-patterns:
1. CNAME hell: DNS has webapp.example.com CNAME webapp-prod.internal.example.com, library canonicalizes to internal name, keytab has external name
2. Load balancer VIPs: Client connects to vip.example.com, canonicalizes to lb-node-01.example.com, keytab has VIP name
3. IPv6 breaks PTR: Client uses IPv6, PTR lookup fails, canonicalization falls back to IP address literal → ticket request for HTTP/2001:db8::1@REALM (invalid)
The genius approach:

Determine what SPN the client will actually request

KRB5_TRACE=/dev/stderr curl --negotiate -u : https://webapp.example.com 2>&1 | grep -i "Getting credentials"

Compare to what's in the keytab

ssh webapp.example.com "klist -ket /etc/krb5.keytab | grep HTTP"

If they don't match, you have three options:

1. Add the canonicalized name to keytab (kadmin: ktadd HTTP/actual-name@REALM)

2. Disable canonicalization in krb5.conf (rdns=false)

3. Create DNS alias records so canonicalization works predictably

Hiring signal:
· Junior: “The service principal is wrong”
· Senior: “I’ll trace what SPN the client is actually requesting with KRB5_TRACE, compare it to the keytab, and determine if this is a DNS canonicalization issue before touching the keytab or KDC”
Documentation:
· https://web.mit.edu/kerberos/krb5-latest/doc/admin/princ_dns.html
· https://web.mit.edu/kerberos/krb5-latest/doc/admin/conf_files/krb5_conf.html#libdefaults

1. Delegation: S4U2Self, S4U2Proxy, and Constrained Delegation
   Why hiring managers care: Modern architectures have middle-tier services calling backend services on behalf of users. Engineers who don’t understand delegation can’t build these systems securely.

The brutal reality:

· Unconstrained delegation (forwardable tickets) = security nightmare, never use in production

· Constrained delegation (S4U2Proxy) = what you actually want, but configuration is painful

· S4U2Self = service obtains ticket for a user without the user's credentials (protocol-transition)

Mental model:

User → WebApp → Database
^ ^
| |
User's TGT WebApp acts as user

Three delegation mechanisms:
1. Forwardable tickets (legacy, dangerous):
· User gets TGT with forwardable flag
· WebApp receives user's TGT, can impersonate user to any service
· Attack: Compromise WebApp = compromise user everywhere
2. S4U2Proxy (constrained delegation):
· WebApp is configured to impersonate users to specific services only
· Requires KDC support, must configure allowed delegation targets
· WebApp gets service ticket for Database, marked as "forwardable" by KDC
3. S4U2Self (protocol transition):
· WebApp authenticates user via non-Kerberos (e.g., OIDC, SAML)
· WebApp asks KDC: "Give me a ticket for user@REALM as if user authenticated"
· Requires TRUSTED_TO_AUTHENTICATE_FOR_DELEGATION flag in AD
Library usage (MIT Kerberos GSS-API):

```
// S4U2Proxy example
OM_uint32 maj_stat;
gss_cred_id_t delegated_cred;
gss_name_t target_service;

// Import service name (e.g., "database/db.example.com@REALM")
gss_import_name(&min_stat, &service_buffer, GSS_C_NT_HOSTBASED_SERVICE, &target_service);

// Acquire credentials with constrained delegation
maj_stat = gss_acquire_cred_impersonate_name(
&min_stat,
webapp_cred, // WebApp's credentials
user_name, // User to impersonate
0, // Lifetime
```

GSS_C_NO_OID_SET, // Mechs
GSS_C_INITIATE,
&delegated_cred, // Output: credentials for user
NULL, NULL
);

Production failure modes:
1. Missing delegation permissions: KDC not configured to allow WebApp → Database delegation → “KDC policy rejects request”
2. Ticket flags wrong: Ticket not marked forwardable → gss_acquire_cred_impersonate_name fails silently
3. Cross-realm delegation: User in REALM1, WebApp in REALM2 → S4U2Proxy doesn’t work (requires same realm or complex trust)
AD-specific gotcha:
· Active Directory uses different attribute names:
· msDS-AllowedToDelegateTo = constrained delegation targets
· TRUSTED_TO_AUTHENTICATE_FOR_DELEGATION = protocol transition permission
· MIT Kerberos KDC doesn’t support S4U extensions natively (need enterprise patches or use AD)
Hiring signal:
· Junior: “Delegation means forwarding tickets”
· Senior: “Delegation in production means S4U2Proxy with explicit allowed targets configured in the KDC, and I know how to verify delegation permissions and trace failures with GSSAPI error codes”
Documentation:
· https://web.mit.edu/kerberos/krb5-latest/doc/appdev/gssapi.html#gss-acquire-cred-impersonate-name
· https://learn.microsoft.com/en-us/windows-server/security/kerberos/kerberos-constrained-delegation-overview

1. GSSAPI vs Direct Kerberos API: When Libraries Abstract Too Much
   Why hiring managers care: Most production services use GSSAPI (HTTP Negotiate, SSH GSSAPI, NFS sec=krb5). Engineers who don’t know when GSSAPI is hiding problems can’t debug failures.
   The layering:

Application (curl, sshd, httpd)
↓
GSSAPI (RFC 2743/2744) ← Generic security abstraction
↓
Kerberos GSS mechanism (libgssapi_krb5.so)
↓

libkrb5.so ← Actual Kerberos protocol

↓

KDC

Critical distinction:

· Direct Kerberos API: krb5_get_init_creds_password(), krb5_get_creds(), etc.

· Full control, explicit error codes

· Used by: kinit, ksu, custom auth tools

· GSSAPI: gss_init_sec_context(), gss_accept_sec_context()

· Mechanism-agnostic (can use Kerberos, NTLM, SPNEGO)

· Error codes are generic and less helpful

· Used by: Apache mod_auth_gssapi, OpenSSH, curl –negotiate

Where GSSAPI hides problems:

GSSAPI error (unhelpful)

gss_init_sec_context: Unspecified GSS failure. Minor code may provide more information

Actual Kerberos error (helpful)

KRB5KDC_ERR_S_PRINCIPAL_UNKNOWN: Server not found in Kerberos database

Debug technique:

Enable GSSAPI debug logging (application-specific)

Apache mod_auth_gssapi:

GssapiLocalName on

GssapiDebug on

curl:

curl --negotiate -u : -v https://example.com 2>&1 | grep -i gss

OpenSSH:

ssh -o GSSAPIAuthentication=yes -vvv user@host 2>&1 | grep -i gss

When to use which:

Scenario	Use GSSAPI	Use Kerberos API
HTTP auth	Yes (mod_auth_gssapi, SPNEGO)	No
SSH auth	Yes (builtin GSSAPI)	No
Custom auth tool	No	Yes
Service-to-service with multiple mechs	Yes	No
Need detailed error codes	No	Yes

Production gotcha:
· GSSAPI can silently fall back to NTLM if Kerberos fails (SPNEGO negotiation)
· You think auth is working with Kerberos, actually using NTLM
· NTLM has weaker security properties
Check which mechanism was actually used:

Client-side

klist # After successful auth, check if service ticket was obtained

Server-side (Apache)

grep -i "gss_accept_sec_context" /var/log/httpd/error_log

Look for: "Using mechanism Kerberos 5" vs "Using mechanism NTLM"

Hiring signal:
· Junior: “GSSAPI is Kerberos”
· Senior: “GSSAPI is a generic API that can use Kerberos as one mechanism, and I know how to trace which mechanism was actually used and translate generic GSS errors to specific Kerberos failures”
Documentation:
· https://web.mit.edu/kerberos/krb5-latest/doc/appdev/gssapi.html
· https://web.mit.edu/kerberos/krb5-latest/doc/appdev/refs/api/index.html

1. Replay Protection and Replay Caches
   Why hiring managers care: “Decrypt integrity check failed” is the #2 Kerberos error message. Half the time it’s replay cache corruption, not actual crypto failures.
   The problem Kerberos solves:
   · Tickets are reusable within their lifetime
   · Without protection, attacker captures ticket → replays it → impersonates user
   · Authenticators provide one-time-use proof of ticket possession
   How it works (library level):

Client → Server: {Ticket, Authenticator}
Ticket: Reusable credential (encrypted with service key)
Authenticator: One-time token (includes timestamp, checksum)

Server:
1. Decrypts ticket with service key
2. Extracts session key from ticket
3. Decrypts authenticator with session key
4. Checks timestamp is within clock skew window
5. Checks authenticator is NOT in replay cache
6. Adds authenticator hash to replay cache

Replay cache storage:
· Default location: /var/tmp/krb5_rcache_* (tmpfs or real disk)

· Format: Binary file with (timestamp, authenticator_hash) entries
· Cleared automatically when entries expire (5 minutes past their timestamp)
Production failure modes:
1. Replay cache corruption:

Symptoms

tail /var/log/httpd/error_log
gss_accept_sec_context: Decrypt integrity check failed

Real cause: Corrupt replay cache

ls -la /var/tmp/krb5_rcache_HTTP*

File has wrong permissions or is corrupted

Fix

rm /var/tmp/krb5_rcache_*
systemctl restart httpd

```
2.  tmpfs fills up:
·   /var/tmp is full → can't write replay cache → all auth fails
·   Silent failure in many implementations
3.  Multi-instance services:
·   Two Apache instances use same replay cache file
·   Race condition on writes → corruption → auth fails randomly
4.  Permission denied:
·   Service runs as httpd user
·   Replay cache owned by root
·   Service can't write → fails with generic decrypt error
```

[libdefaults]
# Change replay cache type
# Types: file, none (dangerous), dfl (default)
default_rcache_name = FILE:/run/httpd/krb5_rcache

The genius debug:

Check if replay cache is the issue

mv /var/tmp/krb5_rcache_HTTP* /tmp/backup/

Try auth again

If it works → replay cache was corrupted

If it still fails → actual decrypt issue (wrong key, kvno mismatch)

Hiring signal:
· Junior: “Decrypt errors mean wrong password”
· Senior: “I’ll check replay cache first because corrupt rcache files cause decrypt errors that have nothing to do with keys, and I know how to identify replay cache vs actual crypto failures”
Documentation:
· https://web.mit.edu/kerberos/krb5-latest/doc/basic/rcache_def.html
· https://web.mit.edu/kerberos/krb5-latest/doc/admin/troubleshoot.html

Section 2: Genius-Level Understanding
The Meta-Framework: Kerberos as a Distributed Capability System
Advanced mental model:
Kerberos is not “authentication”—it’s a distributed capability token system with three primitives:
1. Tickets = bearer capabilities (like AWS STS tokens)
· Prove right to access a resource

· Time-bounded, non-revocable (except via KDC-side lifetime limits)
· Encrypted to prevent forgery, but otherwise stateless
2. The KDC = capability mint
· Single source of truth for “what capabilities can this principal have?”
· Stateless (doesn’t track issued tickets)
· Can’t revoke tickets after issuance (fundamental limitation)
3. Authenticators = proof-of-possession
· Prevent bearer token theft
· One-time use via replay cache
· Bind capability (ticket) to a specific session
Why this matters:
· Engineers think “Kerberos is SSO”—it’s not, it’s a trust graph where KDC is the root of trust
· Common mistake: “I’ll just revoke the user’s ticket”—you can’t, you can only change their password (which invalidates future ticket requests)
· Security implication: Stolen ticket is valid until expiration, period
The trust graph:

```
  KDC (Root of Trust)
   / \
  /   \
 /     \
```

User Service
| |
| ├── Has keytab (long-term key)
| └── Validates tickets using session key
|
└── Has password (derives long-term key)

Compare to other systems:
· JWT: Stateless bearer tokens, but no KDC (issuer signs, holder verifies)
· OAuth 2.0: Capabilities with revocation (authorization server tracks tokens)
· Kerberos: Stateless capabilities, no revocation, but mutual auth and delegation

Production Scenario 1: The “Worked Yesterday” SSO Failure
Situation:
· Enterprise web app uses Kerberos for SSO (mod_auth_gssapi)
· Auth worked fine for 6 months
· Monday morning: 50% of users can’t log in, 50% can
· Logs show: gss_accept_sec_context: Server not found in Kerberos database
Genius-level diagnostic process:
1. Pattern recognition:

· 50/50 split suggests infrastructure change, not user-specific
· “Server not found” means SPN mismatch, not expired tickets
2. Hypothesis generation:

H1: DNS changed over weekend

dig webapp.example.com

Returns: webapp.example.com. 300 IN A 10.0.1.5

webapp.example.com. 300 IN A 10.0.1.6 ← New IP!

H2: Load balancer added, DNS round-robin

for i in {1..10}; do
dig +short webapp.example.com
done

Alternates between 10.0.1.5 and 10.0.1.6

```
3.  Root cause:
·   Friday: Single web server, keytab has HTTP/webapp.example.com@REALM
·   Weekend: Load balancer deployed, DNS now points to LB VIP
·   Clients canonicalize hostname:
·   50% resolve to webapp-lb-01.internal.example.com (new LB)
·   50% still resolve to webapp.example.com (old server)
```

```
·   LB hostname is NOT in keytab
4.  Verification:
```

Test from client machine

KRB5_TRACE=/dev/stderr curl --negotiate -u : https://webapp.example.com 2>&1 | grep "Getting credentials"

Output shows:

Getting credentials user@REALM -> HTTP/webapp-lb-01.internal.example.com@REALM

This is the canonicalized name!

```
5.  Fix:
```

Option A: Add LB hostname to keytab

kadmin: ktadd -k /etc/httpd/http.keytab HTTP/webapp-lb-01.internal.example.com@REALM

Option B: Disable canonicalization (client-side)

/etc/krb5.conf on all clients:

[libdefaults]
rdns = false
dns_canonicalize_hostname = false

Key insight:
· The error message (Server not found) is accurate but misleading
· The “server” is not the physical server, it’s the SPN
· Understanding DNS canonicalization behavior is critical

Production Scenario 2: The “Invisible” Clock Skew
Situation:
· Service-to-service auth fails intermittently (10-20% of requests)
· Both services on same subnet, same NTP server
· date shows identical times on both hosts
· No clock skew errors in logs
Genius-level diagnostic process:
1. Recognition:
· Intermittent failures suggest race condition or threshold behavior
· No explicit clock skew errors → skew is borderline
2. Hypothesis:
· Clock skew is EXACTLY at the 5-minute threshold
· Sub-second timing variations cause 50/50 failures
3. Verification:

Check actual skew (sub-second precision)

ssh service-a "date +%s.%N" # 1706825901.471868
ssh service-b "date +%s.%N" # 1706825601.123456

Difference: 300.348412 seconds (> 300.0 threshold)

```
4.  Root cause:
·   NTP configured but not running (ntpd crashed)
·   Clocks drifted apart slowly over weeks
·   Hit exact threshold where Kerberos rejects ~50% of requests (depending on subsecond
5.  Why “date” was misleading:
```

"date" output:

service-a: Mon Feb 3 14:05:01 UTC 2026
service-b: Mon Feb 3 14:00:01 UTC 2026

Looks like 5 minutes exactly, but humans round

Actual difference: 5 minutes 0.348 seconds → failure

```
6.  The fix:
```

Verify NTP is actually running

systemctl status chronyd

Force immediate sync

chronyc makestep

Monitor skew

watch -n 1 'chronyc tracking | grep "System time"'

Key insight:
· Kerberos enforces skew limits with microsecond precision
· Human-readable time (“14:05”) hides sub-second drift
· Always use date +%s.%N for debugging

Production Scenario 3: The Delegation Nightmare
Situation:
· Three-tier app: User → Web → API → Database
· Web service needs to query Database as the user
· S4U2Proxy configured, works in dev, fails in prod
· Error: KDC policy rejects request (KDC_ERR_POLICY)
Genius-level diagnostic process:
1. Mental model check:

Expected flow:
User → Web: User's TGT
Web → KDC: S4U2Proxy request for API service
KDC → Web: Forwardable ticket for user@REALM → API
Web → API: Use forwarded ticket
API → KDC: S4U2Proxy request for Database
KDC → API: Forwardable ticket for user@REALM → Database

```
2.  Hypothesis: Missing delegation chain:
·   S4U2Proxy requires each hop to be configured
·   Web → API delegation exists
·   API → Database delegation missing
3.  Verification (Active Directory example):
```

Check Web service delegation settings

Get-ADComputer web-server -Properties msDS-AllowedToDelegateTo

Output: HTTP/api.example.com (✓ Correct)

Check API service delegation settings

Get-ADComputer api-server -Properties msDS-AllowedToDelegateTo

Output: (empty) ← Problem!

```
4.  Root cause:
·   Web configured to delegate to API
·   API NOT configured to delegate to Database
·   Two-hop delegation requires both hops configured
5.  Fix:
```

Add Database delegation to API server

Set-ADComputer api-server -Add @{
'msDS-AllowedToDelegateTo' = 'MSSQLSvc/db.example.com:1433'
}

```
6.  But wait, there's more:
·   Even after fixing, still fails
·   New error: KDC_ERR_BADOPTION (ticket flags not acceptable)
7.  Deeper diagnosis:
```

Check ticket flags at each hop

klist # On Web server after receiving user's ticket

Flags: FIA (Forwardable, Initial, preauthenticated)

Request service ticket for API

kvno HTTP/api.example.com
klist

New ticket flags: FA (Forwardable, forwarded) ← Missing 'F'!

```
8.  Actual root cause:
·   User’s initial TGT not marked forwardable
·   S4U2Proxy requires forwardable tickets
·   Dev environment: AD configured to issue forwardable TGTs by default
·   Prod environment: AD configured for non-forwardable TGTs (security policy
9.  Real fix:
```

Option A: Change AD policy (dangerous, affects all users)

Option B: Use S4U2Self (protocol transition) instead

Requires:

Set-ADComputer web-server -Add @{
'UserAccountControl' = 0x1000000 # TRUSTED_TO_AUTHENTICATE_FOR_DELEGATION
}

Key insights:
· Delegation failures rarely have obvious error messages
· Multi-hop delegation requires configuration at every hop
· Ticket flags (forwardable, forwarded, proxiable) control delegation behavior
· S4U2Proxy vs S4U2Self choice depends on whether user's ticket is forwardable

Counterexample Analysis: Configurations That Look Correct But Fail
Counterexample 1: The "Correct" Keytab
Configuration:

Keytab contents

klist -ket /etc/http.keytab
KVNO Principal

```
5 HTTP/webapp.example.com@EXAMPLE.COM (aes256-cts)
5 HTTP/webapp.example.com@EXAMPLE.COM (aes128-cts)
```

Service config (/etc/httpd/conf.d/auth.conf)

AuthType GSSAPI
AuthName "Kerberos Login"
GssapiCredStore keytab:/etc/http.keytab
Require valid-user

Looks perfect, right?

Why it fails:
· Keytab file permissions: rw-r--r-- root:root
· Apache runs as www-data user
· Keytab is readable, so no permission errors in logs
· But on some systems, gss_acquire_cred() requires execute permission on parent directories
The fix:

Not just file permissions

chmod 640 /etc/http.keytab
chown www-data:www-data /etc/http.keytab

But also directory permissions

chmod 755 /etc

Some systems require keytab in specific location

Move to /var/lib/httpd/ and update GssapiCredStore

Lesson:
· “File is readable” doesn’t mean “GSSAPI can read file”
· Library-specific permission requirements matter

Counterexample 2: The “Correct” SPN
Configuration:

DNS

dig webapp.example.com

Answer: 10.0.1.5

Keytab

klist -ket /etc/http.keytab
HTTP/webapp.example.com@EXAMPLE.COM (aes256-cts)

Client test

kinit user@EXAMPLE.COM
kvno HTTP/webapp.example.com@EXAMPLE.COM

Success! Ticket acquired.

But curl --negotiate still fails

Why it fails:

The smoking gun

KRB5_TRACE=/dev/stderr curl --negotiate -u : http://webapp.example.com 2>&1 | grep "Getting credentials"

Output:

Getting credentials user@EXAMPLE.COM -> HTTP/10.0.1.5@EXAMPLE.COM

← Requesting IP address literal, not hostname!

Root cause:
· curl uses IP address if hostname resolution fails
· Library requests ticket for HTTP/10.0.1.5@EXAMPLE.COM
· This SPN doesn’t exist in KDC
The fix:

Check /etc/hosts

cat /etc/hosts

10.0.1.5 webapp ← Short name, not FQDN

Fix DNS resolution

echo "10.0.1.5 webapp.example.com webapp" >> /etc/hosts

Or fix krb5.conf to prevent IP literal usage

[libdefaults]
ignore_acceptor_hostname = false

Lesson:
· Test with the actual client tool, not just kvno
· Library behavior varies by application (curl vs wget vs custom code)

Counterexample 3: The “Correct” KDC Configuration
Configuration:

/etc/krb5.conf

[libdefaults]
default_realm = EXAMPLE.COM

[realms]
EXAMPLE.COM = {
kdc = kdc1.example.com
kdc = kdc2.example.com
kdc = kdc3.example.com
admin_server = kdc1.example.com
}

Why it fails (subtly):
· Multiple KDCs for high availability
· Client sends request to kdc1, gets TGT
· Client sends TGS request to kdc2, gets ticket with different kvno
· Service has kvno=5 (synced from kdc1)

· TGS ticket has kvno=6 (from kdc2, recently updated)
· Auth fails: Decrypt integrity check failed
Root cause:
· Multi-master KDC setup with replication lag
· kvno updated on kdc2, not yet replicated to kdc1
· Client round-robins between KDCs, gets inconsistent state
The fix:

Don't use multiple KDC lines (deprecated)

Use DNS SRV with priority/weight

DNS (weighted SRV records)

_kerberos._tcp.example.com. IN SRV 0 100 88 kdc1.example.com.
_kerberos._tcp.example.com. IN SRV 10 100 88 kdc2.example.com.
_kerberos._tcp.example.com. IN SRV 10 100 88 kdc3.example.com.

krb5.conf (let DNS handle it)

[realms]
EXAMPLE.COM = {
admin_server = kdc1.example.com
}

Omit kdc lines, library uses DNS SRV

Lesson:
· High availability requires KDC state synchronization

· Client-side load balancing can cause inconsistency
· DNS SRV with priority ensures all clients use same primary KDC

Expert-Level Questions: Can You Reason Like a Production Engineer?
Question 1: The Mysterious Timeout
Scenario:

User reports: "Login works on wired network, fails on WiFi"
Your investigation:
- kinit works on both networks
- klist shows valid TGT on both networks
- curl --negotiate to webapp times out on WiFi, works on wired
- No firewall rules blocking WiFi

Question: What’s the most likely root cause, and how do you verify it?

Genius Answer
Root cause: UDP fragmentation on WiFi due to lower MTU. Reasoning: 1. kinit works → TGT acquisition succeeds (small packet) 2. curl --negotiate times out → TGS request or service ticket likely large 3. WiFi MTU typically 1400-1500, wired MTU 1500-9000 4. User with many group memberships → large PAC (Privilege Attribute Certificate) in ticket 5. TGS response exceeds WiFi MTU → fragments → wireless drops fragments → timeout Verification: # Check MTU ip link show wlan0 # mtu 1420 ip link show eth0 # mtu 1500 # Check ticket size kinit user@REALM kvno HTTP/webapp.example.com@REALM klist -e # Check ticket size in cache # Enable TCP fallback (krb5.conf) [libdefaults] udp_preference_limit = 1 # Force TCP for packets > 1 byte Why this is genius: · Recognizes that ticket size varies per user (group memberships) · Understands UDP vs TCP behavior in Kerberos · Knows MTU differences affect network-dependent failures

Question 2: The Phantom kvno Mismatch
Scenario:

Service keytab

klist -ket /etc/krb5.keytab
KVNO Principal
3 host/server.example.com@REALM

Test kinit with keytab - works

kinit -kt /etc/krb5.keytab host/server.example.com@REALM
klist # Shows valid TGT

But service auth fails

Client logs: "Decrypt integrity check failed"

Check KDC

kadmin: getprinc host/server.example.com@REALM
Principal: host/server.example.com@REALM
Key version: 3
kvno matches!

Question: kvno matches everywhere, but decrypt fails. What’s wrong?

Genius Answer
Root cause: Multiple keys with same kvno but different enctypes, and enctype negotiation mismatch. Reasoning: 1. kvno is not unique per key—it’s unique per password/key change 2. When you ktadd a principal, KDC generates keys for all supported enctypes with the same kvno 3. Client and service might negotiate different enctypes Verification: # Full keytab listing with enctypes klist -ket /etc/krb5.keytab KVNO Principal 3 host/server.example.com@REALM (aes256-cts-hmac-sha1-96) 3 host/server.example.com@REALM (aes128-cts-hmac-sha1-96) 3 host/server.example.com@REALM (des3-cbc-sha1) ← Old, unsupported # Check KDC supported enctypes kadmin: getprinc host/server.example.com@REALM Encryption types: aes256-cts-hmac-sha1-96, aes128-cts-hmac-sha1-96 # des3 missing from KDC! # Client requests des3 (old library), KDC issues aes256 # Service tries to decrypt aes256 ticket with des3 key → failure The fix: # Re-key with explicit enctypes kadmin: ktadd -k /etc/krb5.keytab -e aes256-cts,aes128-cts host/server.example.com@REALM # Or update client to prefer modern enctypes [libdefaults] default_tkt_enctypes = aes256-cts aes128-cts default_tgs_enctypes = aes256-cts aes128-cts Why this is genius: · Understands kvno is per password change, not per

enctype · Recognizes enctype negotiation as a failure vector · Knows that keytab can have stale enctypes even with correct kvno

Question 3: The Delegation Paradox
Scenario:

Web service configured for S4U2Proxy:
- User authenticates to Web via Kerberos
- Web impersonates user to API via S4U2Proxy
- API receives ticket, validates it - success

But:
- API calls gss_inquire_cred() to get user's identity
- Returns GSS_S_FAILURE
- Error: "Credentials cache not found"

Why would credential inquiry fail when auth succeeded?

Genius Answer
Root cause: S4U2Proxy provides a ticket, not a credential cache. Reasoning: 1. S4U2Proxy flow: · Web gets service ticket for user@REALM → API · API receives and validates this ticket · API extracts user identity from ticket 2. But gss_inquire_cred() expects: · A credential cache (ccache) with TGT · Or delegated credentials (forwardable TGT) 3. S4U2Proxy gives API a service ticket, not a TGT · API can verify user’s identity · API cannot obtain new tickets on user’s behalf (no TGT) Verification: // What API actually has gss_name_t client_name; gss_display_name(&min, context->src_name, &client_name, NULL); // This works - shows user@REALM // What API is trying gss_cred_id_t delegated_cred; gss_inquire_cred(&min, delegated_cred, &name, NULL, NULL, NULL); // This fails - no credentials, just a ticket The fix: If API needs to act as user to other services, need unconstrained delegation (forwardable TGT): # Client requests forwardable TGT kinit -f user@REALM # Web service accepts delegated credentials # (GSSAPI flag: GSS_C_DELEG_FLAG) # API receives actual TGT for user, can get new service tickets Why this is genius: · Distinguishes between ticket validation and credential delegation · Understands S4U2Proxy gives proof-of-identity, not impersonation capability · Knows when to use constrained (S4U2Proxy) vs unconstrained (forwardable) delegation

Question 4: The Cross-Realm Mystery
Scenario:

Setup:
- User in REALM1
- Service in REALM2
- Cross-realm trust configured:
REALM1: TGS/REALM2@REALM1 exists
REALM2: krbtgt/REALM2@REALM1 exists

User can:
- kinit user@REALM1 - works
- kvno krbtgt/REALM2@REALM1 - works (gets cross-realm TGT)
- kvno HTTP/service.realm2.example.com@REALM2 - works (gets service ticket)

But service auth fails:
- Service logs: "Clock skew too great"
- Both KDCs have correct time (NTP synced)
- User's clock is correct

Question: Why clock skew error when all clocks are synchronized?

Genius Answer
Root cause: Cross-realm ticket timestamps accumulate skew at each hop. Reasoning: 1. Cross-realm path: User → REALM1 KDC → REALM2 KDC → Service 2. Each KDC adds its timestamp: · REALM1 issues cross-realm TGT: timestamp T1 · User presents to REALM2: REALM2 checks T1 against REALM2's clock · REALM2 issues service ticket: timestamp T2 · Service validates: checks T2 against service clock 3. If REALM1 and REALM2 clocks are skewed by 4 minutes: · REALM1 time: 14:00:00 · REALM2 time: 14:04:00 · Cross-realm TGT issued at T1 = 14:00:00 (REALM1) · When user presents to REALM2, REALM2 sees ticket from "past" (14:00 vs 14:04) · Within 5-minute threshold, so REALM2 accepts · REALM2 issues service ticket at T2 = 14:04:00 · Service clock: 14:00:30 (in sync with REALM1) · Service sees ticket from "future" (14:04 vs 14:00:30) = 3.5 minutes skew · Still within 5 minutes, so should work… 4. But: If service clock is also checked against authenticator timestamp: · Client generates authenticator with client time: 14:00:30 · Service ticket timestamp: 14:04:00 · Difference exceeds threshold when accumulated Verification: # Check each KDC's clock ssh realm1-kdc "date -u +%s" # 1706825600 ssh realm2-kdc "date -u +%s" # 1706825840 # Difference: 240 seconds (4 minutes) # Check ticket timestamps klist -e Ticket cache: FILE:/tmp/krb5cc_1000 Valid starting Expires Service principal 02/03/26 14:00:00 02/03/26 14:10:00 krbtgt/REALM2@REALM1 02/03/26 14:04:00 02/03/26 14:14:00 HTTP/service.realm2.example.com@REALM2 # Service ticket issued 4 minutes after cross-realm TGT! The fix: # Synchronize ALL KDCs to same time source # Both REALM1 and REALM2 KDCs must use same NTP servers # Emergency workaround (dangerous): # Increase clock skew tolerance on REALM2 KDC # /var/kerberos/krb5kdc/kdc.conf [kdcdefaults] clockskew = 600 # 10 minutes instead of 5 Why this is genius: · Understands cross-realm introduces multiple timestamp checks · Recognizes that intermediate KDC clock skew compounds · Knows that client, KDC1, KDC2, and service must all be synchronized in cross-realm

Question 5: The Memory Leak That Wasn't
Scenario:

Long-running service (runs for weeks) using GSSAPI:
- Initial auth works fine

- After 3-4 days, auth starts failing randomly
- Restart service → auth works again
- Memory usage normal (no leak)
- Logs show: "Decrypt integrity check failed" after a few days

Service code (simplified):
while (true) {
accept_connection();
gss_accept_sec_context(&min, &context, cred, ...);
// Handle request
gss_delete_sec_context(&min, &context, NULL);
}

Question: What causes periodic auth failures after several days?

Genius Answer
Root cause: Replay cache fills up over time. Reasoning: 1. Replay cache stores authenticator hashes to prevent replay attacks 2. Default replay cache: FILE:/var/tmp/krb5_rcache_ 3. Entries expire after 5 minutes, but: · File grows as entries are added · Expired entries are marked invalid, not deleted · File is never truncated (only on restart) 4. After 3-4 days of high traffic: · Millions of expired entries in rcache file · File lookup becomes O(n) instead of O(1) · Eventually lookup times out → auth fails Verification: # Check rcache file size ls -lh /var/tmp/krb5_rcache_* -rw-------. 1 service service 847M Feb 3 14:00 /var/tmp/krb5_rcache_HTTP # Dump rcache contents (requires compiled tool) ./rcache_dump /var/tmp/krb5_rcache_HTTP | wc -l # 12,847,293 entries (99% expired) The fix: # Option 1: Use in-memory replay cache (requires kernel support) [libdefaults] default_rcache_name = none # Dangerous - disables replay protection! # Option 2: Use directory-based rcache (newer MIT Kerberos) [libdefaults] default_rcache_name = dfl:/run/service/krb5_rcache # Option 3: Periodic cleanup (cron job) 0 */6 * * * find /var/tmp -name 'krb5_rcache_*' -mtime +1 -delete Better service code: // Periodically recreate credentials to force rcache rotation time_t last_cred_refresh = time(NULL); while (true) { accept_connection(); // Refresh credentials every 24 hours if (time(NULL) - last_cred_refresh > 86400) { gss_release_cred(&min, &cred); gss_acquire_cred(&min, GSS_C_NO_NAME, ...); last_cred_refresh = time(NULL); // This causes new rcache file creation } gss_accept_sec_context(&min, &context, cred, ...); gss_delete_sec_context(&min, &context, NULL); } Why this is genius: · Recognizes replay cache as stateful component in “stateless” auth · Understands file-based rcache performance characteristics · Knows that long-running services need rcache management strategy · Connects “decrypt error” to replay cache, not crypto failures

Multiple Perspectives: How Different Engineers See Kerberos
OS/Kerberos Library Implementer View
Core concerns:
· Thread safety of credential cache access
· Performance of cryptographic operations (AES-256 encrypt/decrypt in hot path)

· Backward compatibility with ancient protocols (DES3, RC4)
· Handling network failures gracefully (UDP timeouts, TCP fallback)
Critical implementation details:

// Credential cache locking (MIT Kerberos)
krb5_cc_start_seq_get() // Acquires read lock
krb5_cc_next_cred() // Iterates
krb5_cc_end_seq_get() // Releases lock

// Multi-threaded services MUST:
// 1. Use thread-safe ccache types (not FILE:)
// 2. Synchronize cache writes
// 3. Handle EWOULDBLOCK on cache access

Performance considerations:
· Ticket validation = 1 AES-256 decrypt + 1 HMAC-SHA1 verify
· Typical cost: 10-50 microseconds on modern CPU
· But: Replay cache lookup can be O(n) with large rcache
· Bottleneck is usually rcache, not crypto

Platform/Infra Engineer View
Core concerns:
· How to deploy Kerberos in containerized environments
· Integration with cloud IAM (AWS, GCP, Azure)
· Secrets management (keytab distribution, rotation)
· Multi-region KDC deployment
Container-specific challenges:

Naive approach - broken

FROM ubuntu:22.04
COPY krb5.conf /etc/krb5.conf
COPY service.keytab /etc/krb5.keytab
CMD ["/usr/sbin/httpd"]

Problems:

1. /tmp/krb5_rcache_* in ephemeral container storage → lost on restart

2. Clock skew if host clock != container clock

3. Keytab in image = security hole (baked into image layers)

Correct approach:

FROM ubuntu:22.04

Keytab from secrets manager (runtime injection)

VOLUME /run/secrets

Replay cache on tmpfs

VOLUME /run/rcache
CMD ["krb5-init.sh"] # Init script fetches keytab, sets KRB5RCACHEDIR

Cloud integration:
· AWS: Use AWS Secrets Manager for keytabs, AWS Systems Manager Parameter Store for krb5.conf
· GCP: Workload Identity for service accounts, Secret Manager for keytabs
· Azure: Managed Identity + Key Vault

Security Engineer View
Core concerns:
· Attack surface (replay attacks, credential theft, KDC compromise)
· Encryption strength (weak ciphers still supported)
· Audit logging (who authenticated as whom, when)
· Delegation risks (unconstrained delegation = lateral movement)
Threat model:
1. Credential theft:
· Steal ccache file → impersonate user until ticket expiry
· Mitigation: Use KEYRING: ccache, SELinux policies
2. Keytab compromise:
· Steal keytab → impersonate service forever (until key rotation)
· Mitigation: Hardware security modules (HSM), frequent rotation
3. KDC compromise:
· Game over - attacker can forge any ticket
· Mitigation: KDC on hardened hosts, minimal network exposure, audit logging
4. Delegation attacks:
· Compromise service with unconstrained delegation → steal user TGTs
· Mitigation: Use constrained delegation only, monitor for TGT forwarding
Security best practices:

krb5.conf hardening

[libdefaults]
# Disable weak enctypes
permitted_enctypes = aes256-cts-hmac-sha1-96 aes128-cts-hmac-sha1-96
allow_weak_crypto = false

```
# Enforce PKINIT where possible (certificate-based auth)
pkinit_anchors = FILE:/etc/pki/kerberos/ca.crt
```

Hiring Manager/Interviewer View
What distinguishes levels:

Junior (0-2 years):
· Knows Kerberos exists, has used kinit
· Can follow runbook to fix common issues
· Doesn’t understand why solutions work
Mid-level (2-5 years):
· Debugs auth failures systematically (checks cache, keytab, KDC)
· Understands ticket lifecycle, SPNs, basic delegation
· Can read library documentation and apply it
Senior (5+ years):
· Predicts failure modes from system design
· Understands library implementation details
· Designs secure, scalable Kerberos deployments
· Mentors others on debugging techniques
Staff+ (8+ years):
· Extends/patches Kerberos libraries for custom needs
· Designs cross-realm trust architectures
· Balances security, performance, operability
· Recognized expert who other teams consult
Interview signals:

Question: "Service auth fails, how do you debug?"

Junior: "Check the logs"
Mid: "Check if ticket is expired, verify keytab kvno matches KDC"
Senior: "I'd use KRB5_TRACE to see exact library behavior, verify SPN the client is requesting matches keytab, check for clock skew, and test keytab in isolation before assuming KDC issues"
Staff: "Before debugging, I'd ask about recent changes to DNS, load balancers, or keytab rotation schedules, since auth failures are usually configuration drift. Then I'd trace the full flow from client DNS resolution through ticket acquisition to service validation, checking kvno synchronization, clock skew, and replay cache state at each hop."

Final Wisdom: The Production Mindset
Kerberos fails in production for these reasons (in order):
1. Clock skew (30%)
2. SPN mismatches (25%)
3. Keytab kvno sync issues (20%)
4. DNS/KDC discovery (15%)
5. Replay cache corruption (5%)
6. Actual crypto/protocol bugs (<5%)
The genius engineer knows:
· Always check the basics first (clock, DNS, keytab permissions)
· Use KRB5_TRACE before assuming complex failures

· Understand what the library is actually doing, not what you think it’s doing
· Test at the library level (kinit -kt, kvno) before testing at application level
· Production failures are usually configuration drift, not protocol violations

The hiring bar:

Can you debug a Kerberos failure in production without escalating? If yes, you’re interview-ready.